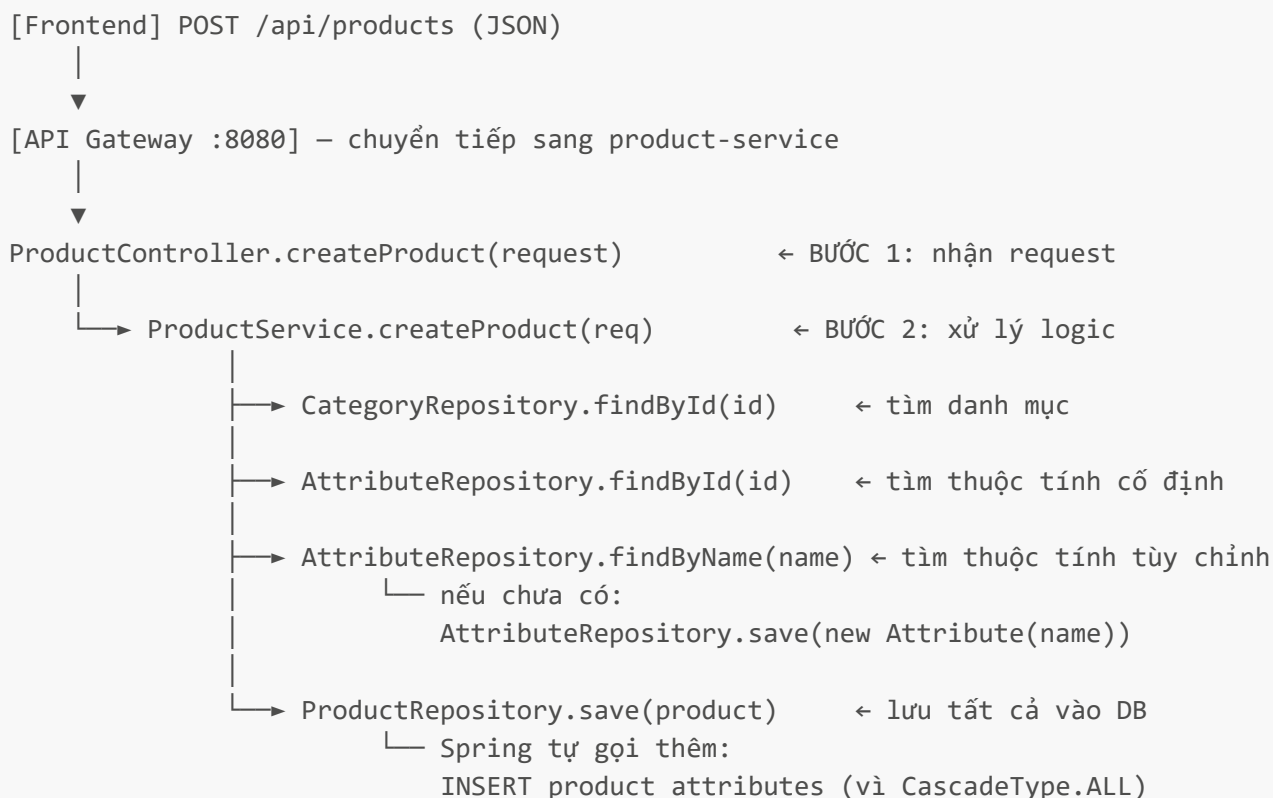


GIẢI THÍCH TOÀN BỘ BACKEND: CHỨC NĂNG THÊM SẢN PHẨM

Giải thích từng file, từng hàm, hàm nào gọi hàm nào

SƠ ĐỒ GỌI HÀM — ĐỌC CÁI NÀY TRƯỚC



FILE 1: `application.properties` — CẤU HÌNH DỊCH VỤ

```
spring.application.name=product-service    # Tên dịch vụ
server.port=8081                           # Chạy ở cổng 8081

# Kết nối database MySQL
spring.datasource.url=jdbc:mysql://localhost:3307/product_db?
createDatabaseIfNotExist=true...
# localhost:3307 → MySQL chạy ở cổng 3307 (Docker)
# product_db → tên database
# createDatabaseIfNotExist=true → nếu chưa có database thì tự tạo
spring.datasource.username=root
spring.datasource.password=1

# Cấu hình JPA/Hibernate
spring.jpa.hibernate.ddl-auto=update
```

```
# "update" nghĩa là: khi khởi động, so sánh Entity với bảng DB hiện có
# Nếu Entity có thêm field mới → tự ALTER TABLE thêm cột
# Nếu Entity bị xóa field → KHÔNG xóa cột (để an toàn)
# Các giá trị khác: create (xóa và tạo lại mỗi lần), none (không làm gì)

spring.jpa.show-sql=false    # Nếu true → in ra mọi câu SQL xuống console (để debug)
```

FILE 2: CorsConfig.java — CHO PHÉP FRONTEND GỌI API

Vấn đề: Trình duyệt có cơ chế bảo mật gọi là CORS. Nếu frontend chạy ở `localhost:3000` mà gọi API ở `localhost:8081` → trình duyệt chặn lại vì khác cổng.

```
@Configuration    // ← Báo Spring: "Đây là file cấu hình, đọc nó khi khởi động"
public class CorsConfig {

    @Bean    // ← Báo Spring: "Tạo object này và quản lý nó (gọi là Bean)"
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/**")    // Áp dụng cho tất cả URL
                    .allowedOriginPatterns("**")    // Cho phép gọi từ bất kỳ
domain nào
                    .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")    //
Cho phép các method này
                    .allowedHeaders("**");    // Cho phép tất cả header
            }
        };
    }
}
```

Hàm `corsConfigurer()` không được gọi thủ công — Spring tự gọi khi khởi động và áp dụng cấu hình này cho mọi request đến.

FILE 3: DataSeeder.java — TỰ ĐỘNG TẠO DỮ LIỆU MẪU

File này chạy **1 lần duy nhất khi ứng dụng khởi động lần đầu** để tạo sẵn dữ liệu mẫu (danh mục, thuộc tính, sản phẩm mẫu).

```
@Component    // Spring quản lý class này
public class DataSeeder implements CommandLineRunner {
    // CommandLineRunner: interface của Spring, yêu cầu implement hàm run()
    // Spring tự gọi run() ngay sau khi ứng dụng khởi động xong
```

Hàm `run()` — chạy tự động khi ứng dụng khởi động

```
@Override
@Transactional
public void run(String... args) {

    // Kiểm tra xem đã có dữ liệu chưa
    if (productRepository.count() > 0) {
        return; // Đã có rồi → thoát, không làm gì
    }
    // productRepository.count() → chạy SQL: SELECT COUNT(*) FROM products
    // Tránh tạo trùng dữ liệu mỗi lần restart server

    // Tạo các thuộc tính
    Attribute thuongHieu = save("Thuong hieu");
    Attribute mauSac      = save("Mau sac");
    // ... tiếp tục tạo các thuộc tính khác

    // Tạo các danh mục kèm thuộc tính
    Category dienThoai = saveCategory("Dien thoai",
        List.of(thuongHieu, mauSac, dungLuong, baoHanh, xuatXu));
    // Danh mục "Điện thoại" có 5 thuộc tính mặc định

    // Tạo sản phẩm mẫu
    addProduct("iPhone 15 Pro Max", new BigDecimal("29990000"), dienThoai,
        List.of(
            new String[]{"Thuong hieu", "Apple"}, // [tên thuộc tính, giá
            new String[]{"Mau sac", "Titan Den"},
            ...
        ));
}
```

Hàm `save(String name)` — tìm hoặc tạo Attribute

```
private Attribute save(String name) {
    return attributeRepository.findByName(name)
        .orElseGet(() -> attributeRepository.save(new Attribute(name)));
    // Dịch: "Tìm Attribute tên 'Thuong hieu' trong DB.
    //       Nếu có rồi → trả về cái đó.
    //       Nếu chưa có → tạo mới và lưu vào DB."
    // Tránh tạo trùng vì Attribute.name có unique = true
}
```

Hàm `saveCategory(String name, List<Attribute> attributes)` — tìm hoặc tạo Category

```
private Category saveCategory(String name, List<Attribute> attributes) {
    return categoryRepository.findByName(name).orElseGet(() -> {
        Category c = new Category(name);    // tạo object Category
        c.setAttributes(attributes);        // gán danh sách thuộc tính mặc định
        return categoryRepository.save(c);   // lưu vào DB
        // Spring tự INSERT vào bảng categories VÀ bảng category_attributes
    });
}
```

Hàm `addProduct(...)` — tạo sản phẩm mẫu

```
private void addProduct(String name, BigDecimal price, Category category,
                        List<String[]> attrPairs) {
    Product p = new Product();
    p.setName(name);
    p.setPrice(price);
    p.setCategory(category);

    List<ProductAttribute> pas = new ArrayList<>();
    for (String[] pair : attrPairs) {
        // pair[0] = tên thuộc tính, pair[1] = giá trị
        Attribute attr = save(pair[0]); // tìm hoặc tạo Attribute
        pas.add(new ProductAttribute(p, attr, pair[1]));
    }
    p.setProductAttributes(pas);
    productRepository.save(p); // lưu product + toàn bộ thuộc tính
}
```

FILE 4: `Category.java` — ENTITY DANH MỤC

```
@Entity
@Table(name = "categories")
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    // IDENTITY = DB tự tăng: 1, 2, 3, 4...
    // Khi INSERT, không cần truyền id, DB tự điền

    @Column(nullable = false, unique = true)
    private String name;
    // nullable = false → NOT NULL trong SQL
    // unique = true → UNIQUE constraint, không được 2 row cùng tên

    @JsonIgnore // ← Khi chuyển Category sang JSON, BỎ QUA field này
    @OneToMany(mappedBy = "category")
```

```

private List<Product> products = new ArrayList<>();
// 1 Category có nhiều Product
// mappedBy = "category" → trỏ đến field "category" trong class Product
// @JsonIgnore vì nếu không: serialize Category → serialize tất cả Product
// → mỗi Product serialize Category → vòng lặp vô tận → StackOverflow

@JsonIgnore
@ManyToMany
@JoinTable(
    name = "category_attributes",           // tên bảng trung gian trong DB
    joinColumns = @JoinColumn(name = "category_id"), // cột của bảng này
    inverseJoinColumns = @JoinColumn(name = "attribute_id") // cột của bảng
    kia
)
private List<Attribute> attributes = new ArrayList<>();
// Quan hệ nhiều-nhiều: 1 danh mục có nhiều thuộc tính, 1 thuộc tính thuộc
nhiều danh mục
// Bảng category_attributes trong DB:
//   category_id | attribute_id
//       1       |       3      ← Điện thoại có thuộc tính Màu sắc
//       1       |       4      ← Điện thoại có thuộc tính RAM
//       2       |       5      ← Laptop có thuộc tính CPU

// ---- Getter/Setter ----
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
// Getter/Setter là bắt buộc trong Java để đọc/ghi field private
// Spring dùng getter để serialize ra JSON
// Spring dùng setter khi tạo object từ JSON
}

```

FILE 5: `Attribute.java` — ENTITY THUỘC TÍNH

```

@Entity
@Table(name = "attributes")
public class Attribute {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name; // "Màu sắc", "RAM", "Xuất xứ"...
    // unique = true → không được 2 thuộc tính cùng tên

    public Attribute() {} // Constructor rỗng – BẮT BUỘC phải có cho JPA
    // JPA/Hibernate khi load từ DB cần constructor rỗng để tạo object trước
    // rồi mới dùng setter để điền dữ liệu vào

    public Attribute(String name) { // Constructor có tham số – tiện khi tạo mới

```

```

        this.name = name;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

FILE 6: `Product.java` — ENTITY SẢN PHẨM

```

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column
    private BigDecimal price;
    // BigDecimal thay vì double/float vì:
    // double: 25000000.0 có thể thành 24999999.9999 (sai số floating point)
    // BigDecimal: chính xác tuyệt đối, dùng cho tiền tệ

    @Column
    private Integer stockQuantity;    // số lượng tồn kho

    @ManyToOne(optional = false)
    @JoinColumn(name = "category_id")
    private Category category;
    // ManyToOne: Nhiều Product → 1 Category
    // optional = false → category KHÔNG được null (NOT NULL)
    // @JoinColumn(name = "category_id") → cột "category_id" trong bảng products
    //   là khóa ngoại trỏ đến bảng categories

    @OneToMany(
        mappedBy = "product",          // field "product" trong ProductAttribute
        // trỏ về đây
        cascade = CascadeType.ALL,      // QUAN TRỌNG – giải thích dưới
        orphanRemoval = true,          // xóa item khỏi list → xóa luôn trong DB
        fetch = FetchType.EAGER        // load Product → load luôn cả danh sách
        // thuộc tính
    )
    private List<ProductAttribute> productAttributes = new ArrayList<>();

    // CascadeType.ALL có nghĩa là:

```

```
// Khi SAVE product → tự SAVE luôn tất cả ProductAttribute trong list
// Khi DELETE product → tự DELETE luôn tất cả ProductAttribute liên quan
// Khi MERGE product → tự MERGE luôn ProductAttribute
// → Chỉ cần gọi productRepository.save(product) 1 lần là đủ
}
```

FILE 7: ProductAttribute.java — ENTITY BẢNG TRUNG GIAN

Đây là bảng nối giữa Product và Attribute, đồng thời lưu giá trị cụ thể.

Bảng products:	Bảng product_attributes:	Bảng attributes:
+----+-----+	+----+-----+-----+-----+	+----+-----+
id name	id product_id attr_id value	id name
+----+-----+	+----+-----+-----+-----+	+----+-----+
5 iPhone ←	1 5 3 Xanh →	3 Màu sắc
+----+-----+	2 5 4 8GB →	4 RAM
	+----+-----+-----+-----+	+----+-----+

```
@Entity
@Table(name = "product_attributes")
public class ProductAttribute {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore // Ẩn field này khỏi JSON response
    @ManyToOne(optional = false)
    @JoinColumn(name = "product_id")
    private Product product;
    // @JsonIgnore vì nếu serialize ProductAttribute → serialize Product
    // → serialize List<ProductAttribute> của Product → vòng lặp vô tận

    @ManyToOne(optional = false)
    @JoinColumn(name = "attribute_id")
    private Attribute attribute; // trỏ đến bảng attributes

    @Column(name = "attr_value") // tên cột trong DB là "attr_value"
    private String value; // giá trị cụ thể: "Xanh titan", "8GB"...

    public ProductAttribute() {} // Constructor rỗng – bắt buộc cho JPA

    // Constructor tiện lợi – dùng trong Service khi tạo mới
    public ProductAttribute(Product product, Attribute attribute, String value) {
        this.product = product;
        this.attribute = attribute;
        this.value = value;
    }
}
```

```
}  
}
```

FILE 8: `CategoryRepository.java` — KHO DANH MỤC

```
public interface CategoryRepository extends JpaRepository<Category, Long> {  
    // JpaRepository<Category, Long>:  
    // Category = entity mà repo này quản lý  
    // Long      = kiểu của khóa chính (id)  
  
    Optional<Category> findByName(String name);  
    // Spring tự tạo câu SQL từ tên method:  
    // "findBy" + "Name" → WHERE name = ?  
    // SQL: SELECT * FROM categories WHERE name = 'Dien thoai'  
    // Trả về Optional vì có thể không tìm thấy  
  
    // Từ JpaRepository, được miễn phí:  
    // .findById(id)      → SELECT * FROM categories WHERE id = ?  
    // .findAll()         → SELECT * FROM categories  
    // .save(category)    → INSERT hoặc UPDATE  
    // .deleteById(id)    → DELETE FROM categories WHERE id = ?  
    // .existsById(id)    → SELECT COUNT(*) > 0 WHERE id = ?  
    // .count()           → SELECT COUNT(*) FROM categories  
}
```

FILE 9: `AttributeRepository.java` — KHO THUỘC TÍNH

```
public interface AttributeRepository extends JpaRepository<Attribute, Long> {  
  
    Optional<Attribute> findByName(String name);  
    // SQL: SELECT * FROM attributes WHERE name = ?  
    // Dùng trong ProductService để tìm thuộc tính tùy chỉnh theo tên  
    // trước khi quyết định có tạo mới không  
}
```

FILE 10: `ProductRepository.java` — KHO SẢN PHẨM

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
  
    // Lấy tất cả sản phẩm, sắp xếp id giảm dần (mới nhất lên đầu), có phân trang  
    Page<Product> findAllByIdDesc(Pageable pageable);  
    // Spring đọc tên method và tự tạo SQL:  
    // SELECT * FROM products ORDER BY id DESC LIMIT ? OFFSET ?  
}
```



```
// Pageable: chứa thông tin trang (page=0, size=20)
// Page<Product>: chứa danh sách + tổng số trang + tổng số phần tử

// Tìm kiếm theo từ khóa trong tên sản phẩm hoặc tên danh mục
@Query("SELECT p FROM Product p WHERE " +
      "LOWER(p.name) LIKE LOWER(CONCAT('%',:keyword,'%')) " +
      "OR LOWER(p.category.name) LIKE LOWER(CONCAT('%',:keyword,'%')) " +
      "ORDER BY p.id DESC")
Page<Product> searchByKeyword(@Param("keyword") String keyword, Pageable
pageable);
// LOWER() = chuyển thành chữ thường → tìm kiếm không phân biệt hoa thường
// CONCAT('%',:keyword,'%') = bao keyword bằng % → tìm bất kỳ vị trí nào
// SQL thực tế: WHERE LOWER(name) LIKE '%iphone%' OR LOWER(category_name) LIKE
'%iphone%'

List<Product> findAllByOrderByIdDesc();
// Giống trên nhưng không phân trang – trả về tất cả sản phẩm
}
```

FILE 11: `CategoryController.java` — CONTROLLER DANH MỤC

```
@RestController
@RequestMapping("/api/categories")
public class CategoryController {

    private final CategoryRepository categoryRepository;
    // Lưu ý: Controller này gọi THẲNG vào Repository, KHÔNG qua Service
    // Vì logic đơn giản (chỉ đọc) nên không cần tạo Service riêng

    public CategoryController(CategoryRepository categoryRepository) {
        this.categoryRepository = categoryRepository;
    }

    // — HÀM 1: Lấy tất cả danh mục —————
    @GetMapping // ← GET /api/categories
    public List<Map<String, Object>> getAllCategories() {
        log.info("GET /api/categories");

        return categoryRepository.findAll() // SELECT * FROM categories
            .stream() // chuyển List thành Stream để xử lý
            .map(c -> Map.<String, Object>.of("id", c.getId(), "name",
            c.getName()))
            // map() = biến đổi mỗi Category thành Map {id: 1, name: "Điện
            thoại"}
            // Không trả về cả object Category vì có @JsonIgnore ở products và
            attributes
            .collect(Collectors.toList()); // gom kết quả lại thành List

        // JSON trả về:
    }
```

```

        // [{"id":1,"name":"Dien thoai"}, {"id":2,"name":"May tinh"}, ...]
    }

    // — HÀM 2: Lấy thuộc tính của 1 danh mục —————
    @GetMapping("/{id}/attributes")    // ← GET /api/categories/1/attributes
    @Transactional(readOnly = true)    // ← Cần @Transactional vì dùng LAZY loading
    public List<Map<String, Object>> getCategoryAttributes(@PathVariable Long id)
    {
        // @PathVariable: lấy giá trị từ URL - /api/categories/1/attributes → id =
        1

        log.info("GET /api/categories/{}/attributes", id);

        Category category = categoryRepository.findById(id)
            .orElseThrow(() -> {
                log.warn("Category not found: id={}", id);    // ghi cảnh báo
                return new IllegalArgumentException("Category không tồn tại");
                // Spring tự chuyển exception này thành HTTP 500
            });

        return category.getAttributes()    // lấy danh sách thuộc tính của danh mục
            // Câu SQL phía sau: SELECT a.* FROM attributes a
            // JOIN category_attributes ca ON a.id = ca.attribute_id
            // WHERE ca.category_id = 1
            .stream()
            .map(a -> Map.<String, Object>.of("id", a.getId(), "name",
a.getName()))
            .collect(Collectors.toList());

        // JSON trả về:
        // [{"id":3,"name":"Mau sac"}, {"id":4,"name":"RAM"}, {"id":5,"name":"Dung
        luong"}]
    }
}

```

Tại sao getCategoryAttributes cần @Transactional?

Mặc định, `Category.attributes` có `@ManyToMany` không có `FetchType.EAGER` → nó là **LAZY** (chỉ load khi cần). Khi gọi `category.getAttributes()` bên ngoài transaction, Hibernate đã đóng session → không load được → exception. `@Transactional` giữ session mở cho đến khi method kết thúc.

FILE 12: ProductController.java — CONTROLLER SẢN PHẨM

```

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {

```

```

        this.productService = productService;
        // Spring tự inject – không cần new ProductService()
        // Gọi là "Dependency Injection" (DI)
    }

    // — HÀM 1: Lấy danh sách sản phẩm (có tìm kiếm + phân trang) —————
    @GetMapping // ← GET /api/products
    public Page<Product> getProducts(
        @RequestParam(required = false) String keyword, // ?
        keyword=iphone (không bắt buộc)
        @RequestParam(defaultValue = "0") int page, // ?page=0 (mặc
        định trang 0)
        @RequestParam(defaultValue = "20") int size) { // ?size=20 (mặc
        định 20 item/trang)

        log.info("GET /api/products keyword='{ }' page={ } size={ }", keyword, page,
        size);
        return productService.getProducts(keyword, page, size);
        // Spring tự serialize Page<Product> thành JSON:
        // {"content": [...], "totalPages": 5, "totalElements": 95, "number": 0,
        ...}
    }

    // — HÀM 2: Lấy 1 sản phẩm theo ID —————
    @GetMapping("/{id}") // ← GET /api/products/5
    public ResponseEntity<Product> getProductById(
        @PathVariable Long id,
        HttpServletRequest request) { // HttpServletRequest: object đại diện
        cho HTTP request

        String cid = request.getHeader("X-Correlation-ID");
        // Lấy header X-Correlation-ID – được feedback-service gửi kèm khi gọi
        inter-service
        // Dùng để biết request này đến từ đâu (tracing)

        log.info("[INTER-SERVICE] [CID:{ }] GET /api/products/{ }", cid != null ?
        cid : "N/A", id);
        return ResponseEntity.ok(productService.getProductById(id));
        // ResponseEntity.ok() = HTTP 200 OK kèm body
    }

    // — HÀM 3: Tạo sản phẩm mới —————
    @PostMapping // ← POST /api/products
    public ResponseEntity<Product> createProduct(@RequestBody Map<String, Object>
    request) {
        // @RequestBody: lấy JSON body của request và parse thành Map<String,
        Object>
        // Ví dụ body: {"name":"iPhone","price":25000000,"categoryId":1,...}
        // Sau parse: {"name":"iPhone", "price":25000000, "categoryId":1, ...}

        log.info("[ADMIN] POST /api/products name='{ }'", request.get("name"));
        Product created = productService.createProduct(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
        // HTTP 201 Created – đúng chuẩn REST hơn 200 OK khi tạo mới resource
    }

```

```

    }

    // — HÀM 4: Xóa sản phẩm —————
    @DeleteMapping("/{id}") // ← DELETE /api/products/5
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        log.info("[ADMIN] DELETE /api/products/{id}", id);
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
        // HTTP 204 No Content – xóa thành công, không có body trả về
    }

    // — HÀM 5: Lấy thông tin thuộc tính theo ID —————
    @GetMapping("/attributes/{id}") // ← GET /api/products/attributes/3
    public ResponseEntity<Attribute> getAttributeById(
        @PathVariable Long id,
        HttpServletRequest request) {

        String cid = request.getHeader("X-Correlation-ID");
        log.info("[INTER-SERVICE] [CID:{id}] GET /api/products/attributes/{id}", cid
        != null ? cid : "N/A", id);
        // Endpoint này được feedback-service gọi để lấy tên thuộc tính

        Attribute attr = productService.getAttributeById(id);
        return attr != null
            ? ResponseEntity.ok(attr) // HTTP 200 + data
            : ResponseEntity.notFound().build(); // HTTP 404 nếu không tìm
        thấy
    }
}

```

FILE 13: ProductService.java — SERVICE (TRÁI TIM CỦA CHỨC NĂNG)

```

@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;
    private final AttributeRepository attributeRepository;

    public ProductService(ProductRepository productRepository,
        CategoryRepository categoryRepository,
        AttributeRepository attributeRepository) {
        this.productRepository = productRepository;
        this.categoryRepository = categoryRepository;
        this.attributeRepository = attributeRepository;
        // Constructor Injection – Spring nhìn vào constructor, tìm Bean phù hợp
        và inject vào
    }

    // — HÀM 1: Tạo sản phẩm mới —————

```

```

@Transactional // ← Bọc toàn bộ hàm trong 1 transaction
public Product createProduct(Map<String, Object> req) {

    // -- Lấy dữ liệu từ Map --
    String name      = (String) req.get("name");
    Number price     = (Number) req.get("price");
    Number stock     = (Number) req.get("stockQuantity");
    Number categoryId = (Number) req.get("categoryId");
    // Dùng Number (lớp cha) thay vì Long/Integer vì JSON parser có thể trả về
    // Integer hoặc Long tùy giá trị → dùng Number an toàn hơn

    // -- Validate --
    if (name == null || name.isBlank())
        throw new IllegalArgumentException("Tên sản phẩm không được trống");
    // .isBlank() = true nếu chuỗi rỗng "" hoặc chỉ có khoảng trắng "   "

    if (price == null)
        throw new IllegalArgumentException("Giá không được trống");
    if (categoryId == null)
        throw new IllegalArgumentException("Danh mục không được trống");

    // -- Tìm Category --
    Category category = categoryRepository.findById(categoryId.longValue())
        .orElseThrow(() -> new IllegalArgumentException("Category không
tồn tại: id=" + categoryId));
    // .longValue() = chuyển Number → long (kiểu primitive)
    // .orElseThrow() = nếu Optional rỗng → ném exception

    // -- Tạo object Product trong RAM --
    Product product = new Product();
    product.setName(name.trim()); // .trim() xóa khoảng trắng đầu/cuối
    product.setPrice(new java.math.BigDecimal(price.toString()));
    // Number → String → BigDecimal (không có cách chuyển trực tiếp)
    product.setStockQuantity(stock != null ? stock.intValue() : 0);
    // Toán tử 3 ngôi: nếu stock != null thì lấy intValue(), không thì dùng 0
    product.setCategory(category);

    // -- Xử lý thuộc tính CỐ ĐỊNH --
    List<ProductAttribute> productAttributes = new ArrayList<>();

    @SuppressWarnings("unchecked") // tắt warning về unchecked cast
    List<Map<String, Object>> fixedAttrs = (List<Map<String, Object>>)
req.get("fixedAttributes");
    // Cast từ Object về List<Map<...>> vì Java không thể kiểm tra kiểu
    generic lúc runtime

    if (fixedAttrs != null) {
        for (Map<String, Object> entry : fixedAttrs) {
            Long attrId = ((Number) entry.get("attributeId")).longValue();
            String value = (String) entry.get("value");

            Attribute attr = attributeRepository.findById(attrId)
                .orElseThrow(() -> new IllegalArgumentException("Attribute
không tồn tại: id=" + attrId));

```

```

        // Thuộc tính cố định PHẢI tồn tại (là thuộc tính của danh mục)
        // Nếu không tìm thấy → báo lỗi ngay

        productAttributes.add(new ProductAttribute(product, attr, value));
        // Tạo object ProductAttribute trong RAM, chưa lưu DB
    }
}

// -- Xử lý thuộc tính TÙY CHỈNH --
@SuppressWarnings("unchecked")
List<Map<String, Object>> extraAttrs = (List<Map<String, Object>>)
req.get("extraAttributes");

if (extraAttrs != null) {
    for (Map<String, Object> entry : extraAttrs) {
        String attrName = (String) entry.get("name");    // "Xuất xứ"
        String value     = (String) entry.get("value");    // "Màu"

        if (attrName == null || attrName.isBlank()) continue;
        // continue = bỏ qua phần tử này, đi sang phần tử tiếp theo
        // Trường hợp: người dùng click "+ Thêm" nhưng không điền tên

        Attribute attr = attributeRepository.findByName(attrName.trim())
            .orElseGet(() -> attributeRepository.save(new
Attribute(attrName.trim())));
        // orElseGet khác orElseThrow:
        // orElseThrow → ném exception nếu không tìm thấy
        // orElseGet   → chạy lambda function để lấy giá trị thay thế
        //
        // Logic: Tìm Attribute tên "Xuất xứ" trong DB
        // Có rồi → dùng cái đó (tái sử dụng)
        // Chưa có → tạo mới và lưu vào DB ngay
(attributeRepository.save()
        // Sau đó dùng Attribute vừa tạo

        productAttributes.add(new ProductAttribute(product, attr, value));
    }
}

// -- Lưu tất cả vào DB --
product.setProductAttributes(productAttributes);
// Gắn danh sách thuộc tính vào product object

Product saved = productRepository.save(product);
// Spring chạy:
// 1. INSERT INTO products (name, price, stock_quantity, category_id)
//    VALUES ('iPhone 15 Pro', 25000000, 100, 1)
// 2. (Vì CascadeType.ALL) Spring tự INSERT từng ProductAttribute:
//    INSERT INTO product_attributes (product_id, attribute_id,
attr_value)
//    VALUES (5, 3, 'Xanh titan'), (5, 4, '8GB'), (5, 7, 'Màu')
// Tất cả trong cùng 1 transaction

log.info("[ADMIN] Product created: id={}, name='{}', category='{}'",

```

```

        saved.getId(), saved.getName(), saved.getCategory().getName());
    return saved;    // trả về Product đã có id từ DB
}

// — HÀM 2: Lấy danh sách sản phẩm có phân trang —————
@Transactional(readonly = true)    // readonly: tối ưu hiệu năng, không lock
row
public Page<Product> getProducts(String keyword, int page, int size) {
    Pageable pageable = PageRequest.of(page, size);
    // PageRequest.of(0, 20) = trang 0, mỗi trang 20 sản phẩm

    if (keyword != null && !keyword.isBlank()) {
        return productRepository.searchByKeyword(keyword.trim(), pageable);
        // Có từ khóa → tìm kiếm
    }
    return productRepository.findAllByOrderByIdDesc(pageable);
    // Không có từ khóa → lấy tất cả, sắp xếp mới nhất lên đầu
}

// — HÀM 3: Lấy 1 sản phẩm theo ID —————
@Transactional(readonly = true)
public Product getProductById(Long id) {
    return productRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Product không tồn
tại: id=" + id));
    // Dùng bởi: ProductController.getProductById() (inter-service call từ
feedback-service)
}

// — HÀM 4: Xóa sản phẩm —————
@Transactional
public void deleteProduct(Long id) {
    if (!productRepository.existsById(id)) {
        throw new IllegalArgumentException("Product không tồn tại: id=" + id);
    }
    productRepository.deleteById(id);
    // Spring chạy:
    // 1. (Vì CascadeType.ALL + orphanRemoval) DELETE FROM
product_attributes WHERE product_id = 5
    // 2. DELETE FROM products WHERE id = 5

    log.info("[ADMIN] Product deleted: id={}", id);
}

// — HÀM 5: Lấy thuộc tính theo ID —————
@Transactional(readonly = true)
public Attribute getAttributeById(Long id) {
    return attributeRepository.findById(id).orElse(null);
    // orElse(null): nếu không tìm thấy → trả null (không ném exception)
    // Controller sẽ kiểm tra null và trả HTTP 404
    // Dùng bởi: ProductController.getAttributeById() (inter-service call từ
feedback-service)
}

```

```
}  
}
```

BẢNG TÓM TẮT: HÀM NÀO GỌI HÀM NÀO

Ai gọi	Gọi hàm nào	Mục đích
Frontend (JS)	GET /api/categories	Lấy danh sách danh mục để hiển thị dropdown
Frontend (JS)	GET /api/categories/1/attributes	Lấy thuộc tính mặc định của danh mục
Frontend (JS)	POST /api/products	Tạo sản phẩm mới
CategoryController.getAllCategories()	categoryRepository.findAll()	Lấy tất cả danh mục từ DB
CategoryController.getCategoryAttributes()	categoryRepository.findById()	Tìm danh mục theo ID
CategoryController.getCategoryAttributes()	category.getAttributes()	Lấy thuộc tính của danh mục
ProductController.createProduct()	productService.createProduct()	Chuyển cho Service xử lý
ProductService.createProduct()	categoryRepository.findById()	Tìm danh mục
ProductService.createProduct()	attributeRepository.findById()	Tìm thuộc tính cố định
ProductService.createProduct()	attributeRepository.findByName()	Tìm thuộc tính tùy chỉnh

Ai gọi	Gọi hàm nào	Mục đích
ProductService.createProduct()	attributeRepository.save()	Tạo thuộc tính mới nếu chưa có
ProductService.createProduct()	productRepository.save()	Lưu sản phẩm + thuộc tính
DataSeeder.run()	save(name)	Tìm hoặc tạo Attribute
DataSeeder.run()	saveCategory(name, attrs)	Tìm hoặc tạo Category
DataSeeder.run()	addProduct(...)	Tạo sản phẩm mẫu
feedback-service	GET /api/products/attributes/3	Lấy tên thuộc tính